

Skills: A Field Guide

where they live, how to arrange one in a repo, and how to write the SKILL.md

A **skill** is a folder containing a file named **SKILL.md** — a conditional prompt with a header. Every session, Claude loads just the headers of every skill it can see (a map of your prompt library); when your request matches a description, the full body is read in and followed. Header always, body on trigger, bundled files only if needed: **progressive disclosure**.

1 · WHERE SKILLS LIVE — THE CLOUD-VS-LOCAL QUIRK

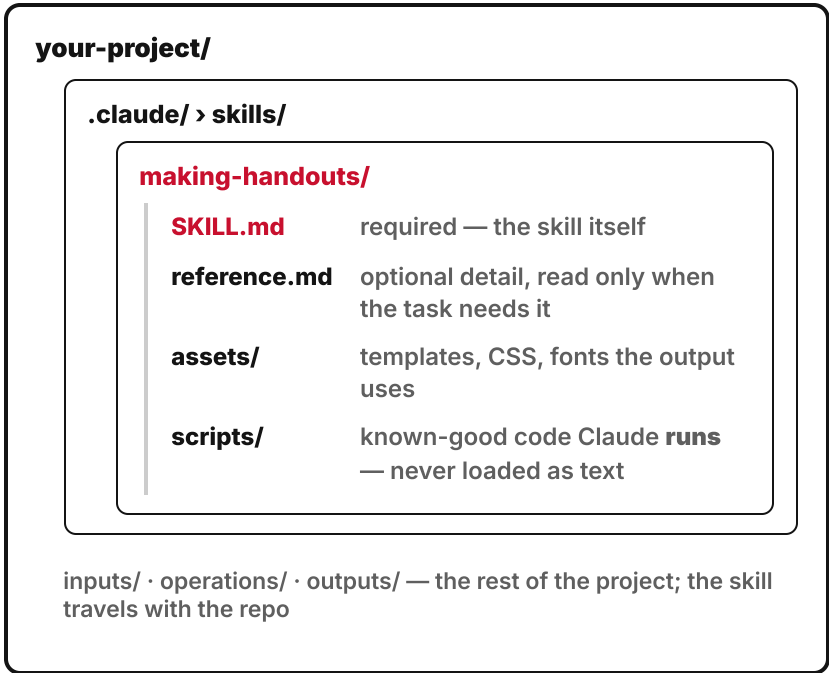
Two completely separate stores, and **they do not sync**. One follows your claude.ai account in the cloud; the other is plain folders on your computer.

	CHAT	COWORK	CODE
Cloud skills uploaded to your claude.ai account · Settings → Capabilities	YES	YES	NO
Local skills ~/claude/skills (personal, every project) · .claude/skills (inside a repo)	NO	YES	YES

The quirk: a skill uploaded to claude.ai does not exist on your machine, and a skill in **~/claude/skills** does not exist in the Chat panel. If a skill "disappears" when you switch panels, this split is why. Keep the canonical copy in a repo; treat any cloud upload as a copy.

2 · HOW A SKILL SITS IN A REPO

One folder per skill, lowercase-with-hyphens (gerunds read well: *making-handouts*, *processing-pdfs*).



- **SKILL.md is the only required file.** Everything else is optional support.
- **Link reference files from SKILL.md** ("See reference.md") and keep them **one level deep** — don't chain file → file → file.
- **scripts/ is for consistency.** Code you've gotten working once runs identically every time, instead of being re-invented per session.

4 · WHAT "FIRING" MEANS

At session start only the headers load — every name and description, none of the bodies. A skill **fires** when its full body is loaded into the conversation and its instructions take effect. Two routes in:

You fire it. Type the slash command (*/making-handouts*) or name it in a prompt ("run the syllabus-redesign skill on this file"). The body loads because you said so.

Claude fires it. You describe the task in plain language; Claude matches it against the descriptions on its map and proposes the skill. You don't need to know it exists.

Rule of thumb: paste the same prompt twice → make it a skill. Same code twice → give it a script. Body outgrows a screen → split into reference files. And write the description for route two: a vague one ("helps with documents") matches nothing, so the skill only ever runs when called by name.

3 · ANATOMY OF THE SKILL.MD

Two zones: a YAML header between --- lines, then a markdown body.

YAML FRONTMATTER — ALWAYS LOADED

name: making-handouts **required** · ≤64 chars · lowercase + hyphens

description: *Formats class notes into the course handout style. Use when the user asks for a handout, printable, or one-pager.* **required** · ≤1,024 chars · third person · must say **what it does and when to use it** — this is all Claude sees before deciding to open the skill

MARKDOWN BODY — LOADED ONLY WHEN TRIGGERED

- **Keep it under ~500 lines;** past that, split into reference files.
- **Assume Claude is already smart.** Write only what it doesn't know: your preferences, conventions, domain specifics.
- **Set the degree of freedom.** Judgment helps → prose steps. Fragile operation → "run exactly *scripts/build.py*, do not modify."
- **One term per concept,** no time-sensitive instructions.
- **Iterate.** Watch what Claude does with it; open it, argue with it, edit.